Дранцев Александр Павлович

С23-722

Лабораторная работа № 1

Задание 1.

Создайте таблицу staff (сотрудники), содержащую следующие поля: id (идентификатор), last_name (фамилия), first_name (имя), second_name (отчество), sex (пол), birthday (дата рождения), post (должность), department (отдел), head_id (идентификатор начальника). Поле id предназначено для хранения числа максимально допустимой длины. Оно является ключом таблицы. Поля last_name, first_name и second_name являются текстовыми полями длиной в 64 байта. Первые два из них не могут быть пусты. Поле пол предназначено для хранения одного символа, не может быть пустым, и допускает только значения «м» и «ж». Поля post и department являются текстовыми полями длиной в 128 байт. Их значения не могу быть пусты и должны образовывать уникальные пары. Поле birthday предназначено для хранения даты и не может быть пустым. Поле head_id относится к тому же типу данных, что и поле id. Оно также является внешним ключом, ссылающимся на поле id той же самой таблицы staff.

Решение

```
CREATE TABLE staff (
   id bigint PRIMARY KEY,
   last_name varchar(64) NOT NULL,
   first_name varchar(64) NOT NULL,
   second_name varchar(64),
   sex char(1) NOT NULL CHECK (sex IN ('м', 'ж')),
   birthday date NOT NULL,
   post varchar(128) NOT NULL,
   department varchar(128) NOT NULL,
            UNIQUE (post, department),
   head_id integer,
   FOREIGN KEY (head_id) REFERENCES staff(id)
);
```

Задание 2.

Создайте убывающую последовательность staff_id_seq.
Заполните таблицу staff 5 строками так, чтобы в поле id размещалось
новое сгенерированное число последовательности, а значения остальных
полей были следующими:

Решение:

```
CREATE SEQUENCE staff_id_seq
   START WITH 1000
   INCREMENT BY -1
   maxvalue 1201;

INSERT INTO staff (id, last_name, first_name, second_name, sex, birthday, post, department, head_id)
VALUES
            (nextval('staff_id_seq'), 'Один', 'Один', 'Один', 'м', '1800-12-21', 'Уборщик', 'СССР', 996),
```

(nextval('staff_id_seq'), 'Два', 'Два', 'Два', 'м', '1800-12-21', 'Уборщик', 'РСФСР', 1000),
(nextval('staff_id_seq'), 'Три', 'Три', 'Три', 'м', '1800-12-21', 'Дворник','РСФСР', 999),
(nextval('staff_id_seq'), 'Четыре', 'Четыре', 'Четыре', 'м', '1800-12-21', 'Дворник', 'СССР', 998),
(nextval('staff_id_seq'), 'Пять', 'Пять', 'Пять', 'м', '1800-12-21', 'Старший уборщик','СССР', 997),
(nextval('staff_id_seq'), 'Сталин', 'Иосиф', 'Виссарионович', 'м', '1879-12-21', 'Председатель', 'ГКО', NULL),
(nextval('staff_id_seq'), 'Молотов', 'Вячеслав', 'Михайлович', 'м', '1890-03-09', 'Заместитель председателя', 'ГКО', NULL),
(nextval('staff_id_seq'), 'Маленков', 'Георгий', 'Максимилианович', 'м', '1902-01-08', 'Начальник', 'УК ЦК ВКП(б)', NULL),
(nextval('staff_id_seq'), 'Ворошилов', 'Климент', 'Ефремович', 'м', '1881-02-04', 'Председатель', 'КО СНК', NULL),
(nextval('staff_id_seq'), 'Микоян', 'Анастас', 'Иванович', 'м', '1895-11-25', 'Председатель', 'КП-ВС РККА', NULL);
UPDATE staff SET head_id = (SELECT id from staff where last_name = 'Сталин') where last_name = 'Молотов';
UPDATE staff SET head_id = (SELECT id from staff where last_name = 'Молотов') where last_name = 'Маленков' OR last_name = 'Ворошилов' OR last_name = 'Микоян';

Задание 3

Напишите запрос, выбирающий из таблицы фамилию, имя, отчество, должность каждого человека, а также фамилию, имя, отчество и должность их руководителей

Решение

select s.last_name,s.first_name,s.second_name,s.post,h.last_name,h.first_name,h.second_name,h.post from staff s join staff h ON s.head_id = h.id;

Задание 4

Напишите запрос, заменяющий значения полей id у всех строк, для которых нет подчинённых, на аналогичные числа, меньшие на 4

Решение

UPDATE staff SET id = id - 4 WHERE id NOT IN (SELECT DISTINCT head_id FROM staff WHERE head_id IS NOT NULL);
select * from staff;

Лабораторная работа № 2

Задание 1.

Описание пациентов больницы. Включает в себя: карты пациентов,

больничные палаты, список амбулаторных периодов пребывания пациентов в

больнице, результаты обследований, операций и других лечебных или

диагностических действий. Карты пациентов состоят из фамилии, имени,

отчества, номера, номера страхового полиса (если есть), указания, платно лечится больной или нет, даты создания, адреса пациента (включая индекс, город, улицу, дом, корпус, квартиру, телефоны), паспортных данных пациента в виде 4-х составляющих, информации о наличии у пациента аллергии на лекарственные препараты (в виде одного большого текста). Больничные палаты состоят из этажа, номера, отделения, вмещаемого количества больных. Список амбулаторных периодов пребывания пациентов в больнице состоит из указания пациента, даты поступления, причин поступления, даты выписки, выписного эпикриза, указания палаты, в которой пациент был размещён. Результаты обследований, операций и других лечебных или диагностических действий состоят из даты действия, названия действия, причин действия, первичного осмотра больного, результатов действия, назначений для последующего лечения, заключения. Больничные палаты могут относиться к нескольким отделениям. Принадлежность больного к отделению определяется в амбулаторном периоде.

Решение

```
DROP TABLE IF EXISTS public."patient's card" CASCADE;
DROP TABLE IF EXISTS public."patient_address" CASCADE;
DROP TABLE IF EXISTS public."patient_passport" CASCADE;
DROP TABLE IF EXISTS public."ill_period" CASCADE;
DROP TABLE IF EXISTS public."hospilal_room_list" CASCADE;
DROP TABLE IF EXISTS public."check_up_results" CASCADE;
DROP TABLE IF EXISTS public."department_list" CASCADE;
DROP TABLE IF EXISTS public."room_department" CASCADE;
DROP TABLE IF EXISTS public."hospital_room_list" CASCADE;

CREATE TABLE IF NOT EXISTS public."patient's card"
(
    patient_id serial,
```

```
    last_name character varying(64) NOT NULL,
    first_name character varying(64) NOT NULL,
    second_name character varying(64) NOT NULL,
    insur_policy character varying(64),
    is_paying boolean NOT NULL,
    date_reg timestamp without time zone NOT NULL,
    allergic text,
    PRIMARY KEY (patient_id)
);

CREATE TABLE IF NOT EXISTS public.patient_address
(
    patient_id integer,
    post_index character varying(64) NOT NULL,
    city character varying(64) NOT NULL,
    street character varying(64) NOT NULL,
    house character varying(64) NOT NULL,
    wing character varying(64) NOT NULL,
    flat character varying(64) NOT NULL,
    phone_number character varying(64) NOT NULL,
    PRIMARY KEY (patient_id),
    CONSTRAINT unique_person UNIQUE (post_index, city, street, house, wing, flat, phone_number)
);

CREATE TABLE IF NOT EXISTS public.patient_passport
(
    patient_id integer,
    series character varying(64) NOT NULL,
    "number" character varying(64) NOT NULL,
    vydan character varying NOT NULL,
    date_vydachi timestamp without time zone NOT NULL,
    PRIMARY KEY (patient_id),
```

```
    UNIQUE (series, "number", vydan, date_vydachi)
);

CREATE TABLE IF NOT EXISTS public.ill_period
(
    ill_period_id serial,
    patient_id integer NOT NULL,
    date_check_in timestamp without time zone NOT NULL,
    reasons text NOT NULL,
    date_check_out timestamp without time zone,
    results text,
    hospital_room_id integer NOT NULL,
    department_id integer NOT NULL,
    PRIMARY KEY (ill_period_id)
);

CREATE TABLE IF NOT EXISTS public.hospital_room_list
(
    hospital_room_id integer NOT NULL,
            hospital_room_number character varying(64) NOT NULL,
    floor_number integer NOT NULL,
    max_capacity integer NOT NULL,
    PRIMARY KEY (hospital_room_id)
);

CREATE TABLE IF NOT EXISTS public.check_up_results
(
    ill_period_id integer,
    date_check_up date,
    action_name text,
    reasons_check_up text,
    review text,
```

```
    results text,
    prescription text,
    conclusion text,
    PRIMARY KEY (ill_period_id,date_check_up,action_name)
);

CREATE TABLE IF NOT EXISTS public.department_list
(
    department_id serial,
    department_name character varying(64) NOT NULL,
    PRIMARY KEY (department_id),
    CONSTRAINT "unique name" UNIQUE (department_name)
);

CREATE TABLE IF NOT EXISTS public.room_department
(
    hospital_room_id integer,
    department_id integer,
    PRIMARY KEY (hospital_room_id, department_id)
);

ALTER TABLE IF EXISTS public.patient_address
    ADD FOREIGN KEY (patient_id)
    REFERENCES public."patient's card" (patient_id) MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE NO ACTION
    NOT VALID;


ALTER TABLE IF EXISTS public.patient_passport
    ADD FOREIGN KEY (patient_id)
    REFERENCES public."patient's card" (patient_id) MATCH SIMPLE
```

```
    ON UPDATE NO ACTION
    ON DELETE NO ACTION
    NOT VALID;


ALTER TABLE IF EXISTS public.ill_period
    ADD FOREIGN KEY (patient_id)
    REFERENCES public."patient's card" (patient_id) MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE NO ACTION
    NOT VALID;


ALTER TABLE IF EXISTS public.ill_period
    ADD FOREIGN KEY (hospital_room_id, department_id)
    REFERENCES public.room_department (hospital_room_id, department_id) MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE NO ACTION
    NOT VALID;


ALTER TABLE IF EXISTS public.check_up_results
    ADD FOREIGN KEY (ill_period_id)
    REFERENCES public.ill_period (ill_period_id) MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE NO ACTION
    NOT VALID;


ALTER TABLE IF EXISTS public.room_department
    ADD FOREIGN KEY (hospital_room_id)
    REFERENCES public.hospital_room_list (hospital_room_id) MATCH SIMPLE
```

```
   ON UPDATE NO ACTION
   ON DELETE NO ACTION
   NOT VALID;


ALTER TABLE IF EXISTS public.room_department
   ADD FOREIGN KEY (department_id)
   REFERENCES public.department_list (department_id) MATCH SIMPLE
   ON UPDATE NO ACTION
   ON DELETE NO ACTION
   NOT VALID;
```

Лабораторная работа № 3

Задание 0.

Выберите всех карты больных (номера), проходивших в текущем месяце обследование «ЭКГ» (название действия)

Решение

```
--0
select patient_id from public."patient's card" JOIN public.ill_period using (patient_id)
JOIN check_up_results using (ill_period_id)
WHERE action_name = 'ЭКГ'
AND EXTRACT(MONTH FROM date_check_up) = EXTRACT (MONTH FROM CURRENT_DATE)
AND EXTRACT(YEAR FROM date_check_up) = EXTRACT (YEAR FROM CURRENT_DATE);
```

Задание 1.

Выберите суммарное количество «платных» больных, размещённых на текущий момент в палатах больницы

Решение

```
--1
select count(patient_id) AS "amount of paying patients stationed in hospital" from public."patient's card" JOIN public.ill_period using (patient_id)
WHERE is_paying = true
AND DATE(date_check_out) >= CURRENT_DATE;
```

Задание 2.

Выберите последний пять назначений больного с ФИО Иванов Иван Иванович.

Решение

```
--2
SELECT mr.*
FROM public."patient's card" pc
JOIN public.ill_period pi
    ON pc.patient_id = pi.patient_id
JOIN public.check_up_results mr
    ON pi.ill_period_id = mr.ill_period_id
WHERE pc.last_name   = 'Иванов'
  AND pc.first_name  = 'Иван'
  AND pc.second_name = 'Иванович'
ORDER BY mr.date_check_up DESC
LIMIT 5;
```

Задание 3

Удалите карты всех больных, не лежащих в палатах и не проходивших обследования в последние пять лет.

Решение

```
DELETE FROM public."patient's card" pc
WHERE
  NOT EXISTS (
    SELECT 1
    FROM public.ill_period pi
    WHERE pi.patient_id = pc.patient_id
      AND CURRENT_DATE BETWEEN pi.date_check_in AND pi.date_check_out
  )
  AND NOT EXISTS (
    SELECT 1
    FROM public.ill_period pi
    JOIN public.check_up_results mr ON mr.ill_period_id = pi.ill_period_id
    WHERE pi.patient_id = pc.patient_id
      AND mr.date_check_up >= CURRENT_DATE - INTERVAL '5 years'
  );
```

Задание 4.

Измените во всех назначениях обследований, содержащих название лекарства «альбуцид», слово «альбуцид» на «сульфацетамид». Например, если поле «назначение» содержало строку «капать альбуцид 3 раза в день по 3 капли», то она должна быть заменена на строчку «капать сульфацетамид 3 раза в день по 3 капли»

Решение

```
UPDATE public.check_up_results
SET prescription = REPLACE(prescription, 'альбуцид', 'сульфацетамид')
WHERE prescription ILIKE '%альбуцид%';
```

Задание 5.

Добавьте в базу данных информацию о стоимости обследования.

Решение

```
ALTER TABLE public.check_up_results
  ADD COLUMN cost numeric(10,2);
```

Задание 6.

Добавьте в базу данных ограничение целостности, не позволяющее выставлять стоимость обследования больше 100000 руб. и меньше 10 руб.

Решение

```
ALTER TABLE public.check_up_results
  ADD CONSTRAINT check_medical_cost
  CHECK (cost >= 10 AND cost <= 100000);
select * from public.check_up_results;
```

Задание на защиту.

выбрать одним запросом больных, кол-во амбулаторных периодов и чекапов

Решение

```
select last_name, first_name, second_name, count(DISTINCT ill_period_id) as "амб. периоды",
count(date_check_up) as "обследования" FROM public."patient's card"
JOIN public.ill_period using (patient_id)
JOIN public.check_up_results using (ill_period_id) GROUP BY (last_name, first_name, second_name);
```